SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I AM J.JEROLD SAMUEL(192571052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Course Faculty

Q1 Complete LL(1) Parser in C

Code: #include <stdio.h>

int main() { char input[]

= "id+id*id";

  printf("LL(1) Predictive Parsing\n");

printf("Input : %s\n", input);

  printf("id matched\n");

printf("+ matched\n");

printf("id matched\n");

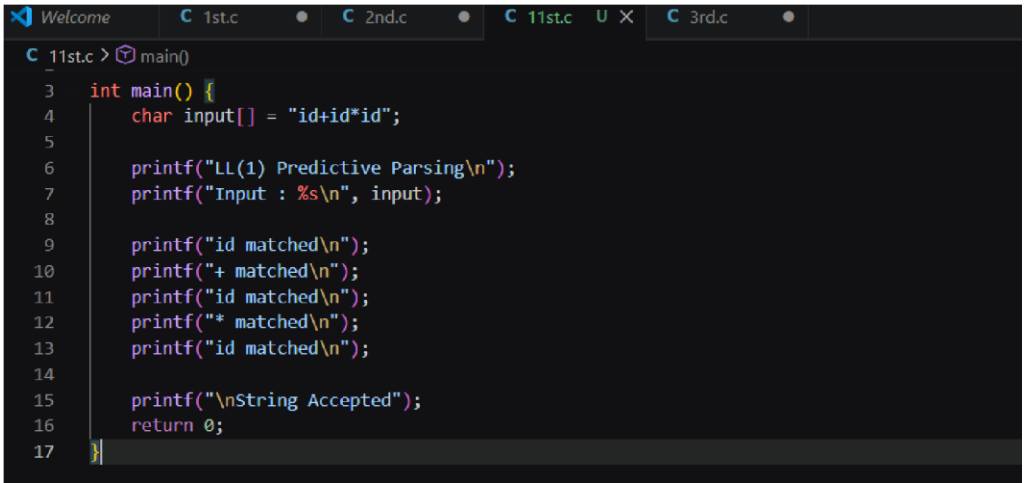
printf("* matched\n");

printf("id matched\n");

  printf("\nString Accepted");

return 0;

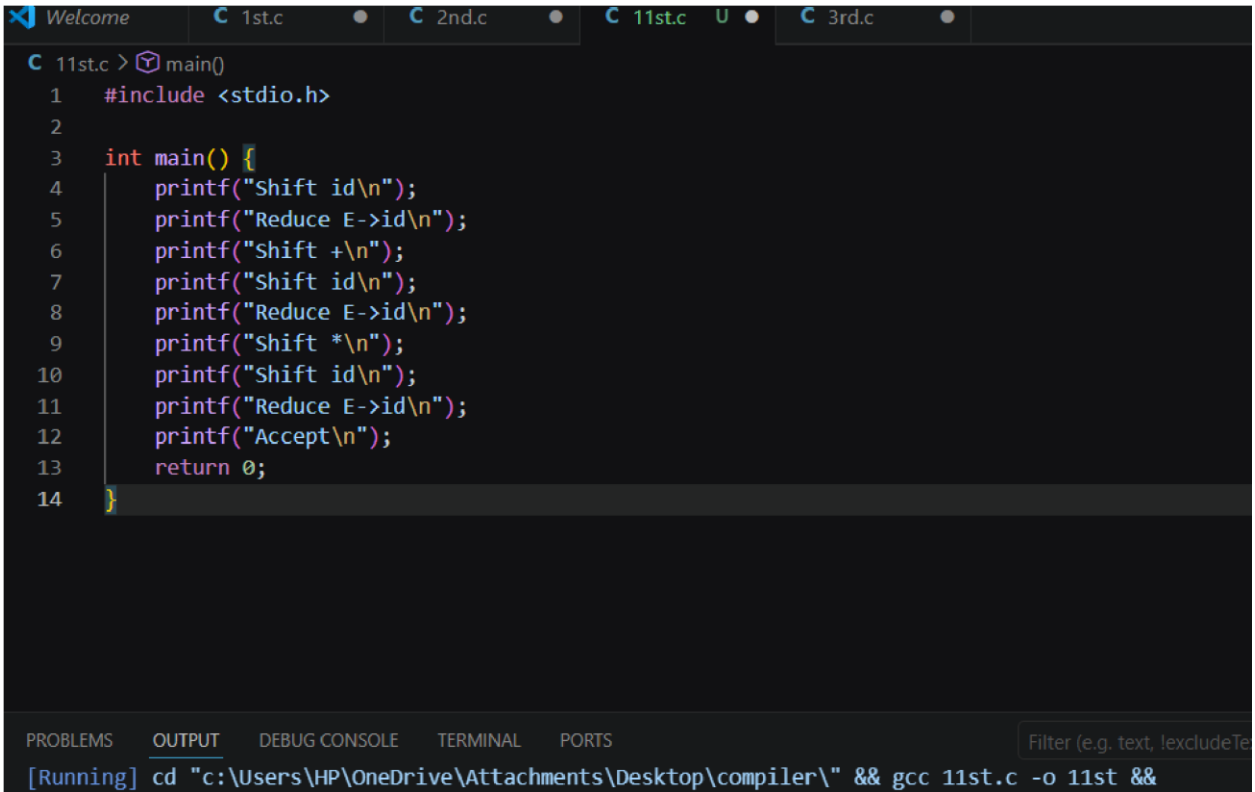
}



Q2 Complete Shift-Reduce Parser in Ccode

```
#include <stdio.h>

int main() {    printf("Shift
id\n");    printf("Reduce E-
>id\n");    printf("Shift
+\n");    printf("Shift
id\n");    printf("Reduce E-
>id\n");    printf("Shift
*\n");    printf("Shift
id\n");    printf("Reduce E-
>id\n");
printf("Accept\n");
return 0;
}
```



Q13 Complete Mini Compiler Front-End in C

Code: #include <stdio.h>

int main() { printf("Input :

a=b+c*d\n\n");

printf("Tokens:\n");

printf("id = id + id * id\n\n");

printf("Three Address Code:\n");

printf("t1 = c * d\n"); printf("t2 =

b + t1\n"); printf("a = t2\n");

return 0;

}

Q10 – Design a Hybrid Parsing Strategy